

Amazon Interview Preparation — Database & SQL Roles

1. Company Profile

Engineering Culture

- **Leadership Principles (LPs)** are the core of every interview — not just behavioral, but technical decisions too
- Data-driven culture: every design decision must be backed by metrics and customer impact
- "Day 1" mentality: bias for action, ownership, frugality with resources
- Two-pizza team ownership: you own your database end to end — schema, tuning, backup, failover

Database Technology Stack

Layer	Technology
OLTP Primary	Aurora PostgreSQL, Aurora MySQL
Data Warehouse	Amazon Redshift
Key-Value / Cache	DynamoDB, ElastiCache (Redis)
Search	Amazon OpenSearch
Time-Series	Amazon Timestream
In-Memory	ElastiCache Memcached
Graph	Amazon Neptune
Ledger	Amazon QLDB
Migration	AWS DMS, Schema Conversion Tool

What Amazon Values in DB/SQL Candidates

- Deep ownership: "You built it, you run it"
 - Customer obsession: designs that minimize latency for end users
 - Operational excellence: monitoring, runbooks, disaster recovery
 - Frugality: right-sizing, partitioning strategies, not over-engineering
 - Dive Deep LP: can explain execution plans, MVCC internals, WAL behavior
-

2. Interview Process

Typical Rounds (SDE II / Data Engineer / SDE III)

Round	Format	Duration	Focus
Phone Screen	HackerRank or CodeSignal	60-75 min	1-2 SQL/coding problems
Technical Phone	Video call with engineer	60 min	SQL + system design lite

Virtual Onsite Loop	4–5 panels via Chime	45 min each	LP + technical deep dives
Bar Raiser Round	Senior/Principal engineer	60 min	LP + deep technical

Onsite Loop Breakdown

- **Round 1:** SQL coding (2–3 medium/hard queries)
- **Round 2:** Database design (schema + indexing + scalability)
- **Round 3:** System design with data focus (e.g., design order management)
- **Round 4:** Behavioral (LP focus, 2–3 LPs per round)
- **Round 5 (Bar Raiser):** Mixed — technical + LP + culture fit

3. SQL Coding Round — 25 Questions with Solutions

Warm-Up (Easy)

Q1. Find the second highest salary in the employees table.

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Alternative using DENSE_RANK
SELECT salary
FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) ranked
WHERE rnk = 2;
```

Q2. List all departments with more than 5 employees.

```
SELECT department_id, COUNT(*) AS emp_count
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5
ORDER BY emp_count DESC;
```

Q3. Find customers who placed orders in both January and February 2024.

```
SELECT customer_id
FROM orders
WHERE DATE_TRUNC('month', order_date) IN ('2024-01-01', '2024-02-01')
GROUP BY customer_id
HAVING COUNT(DISTINCT DATE_TRUNC('month', order_date)) = 2;
```

Q4. Calculate running total of sales per day.

```
SELECT
    sale_date,
```

```

        daily_sales,
        SUM(daily_sales) OVER (ORDER BY sale_date ROWS UNBOUNDED PRECEDING) AS
running_total
FROM (
    SELECT DATE(created_at) AS sale_date, SUM(amount) AS daily_sales
    FROM orders
    GROUP BY DATE(created_at)
) daily;

```

Q5. Find all employees who earn more than their manager.

```

SELECT e.name AS employee, e.salary AS emp_salary,
       m.name AS manager, m.salary AS mgr_salary
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;

```

Intermediate

Q6. Find the top 3 products by revenue per category.

```

WITH ranked AS (
    SELECT
        category_id,
        product_id,
        SUM(quantity * unit_price) AS revenue,
        RANK() OVER (PARTITION BY category_id ORDER BY SUM(quantity * unit_price)
DESC) AS rnk
    FROM order_items oi
    JOIN products p USING (product_id)
    GROUP BY category_id, product_id
)
SELECT category_id, product_id, revenue
FROM ranked
WHERE rnk <= 3;

```

Q7. Identify customers with no orders in the last 90 days but at least one order ever.

```

SELECT c.customer_id, c.email, MAX(o.order_date) AS last_order
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.email
HAVING MAX(o.order_date) < CURRENT_DATE - INTERVAL '90 days'
    OR MAX(o.order_date) IS NULL
    -- but they have at least one order ever
    AND COUNT(o.order_id) > 0;

-- Cleaner version
SELECT c.customer_id, c.email
FROM customers c

```

```

WHERE c.customer_id IN (SELECT DISTINCT customer_id FROM orders)
AND c.customer_id NOT IN (
    SELECT DISTINCT customer_id
    FROM orders
    WHERE order_date >= CURRENT_DATE - INTERVAL '90 days'
);

```

Q8. Calculate month-over-month revenue growth percentage.

```

WITH monthly AS (
    SELECT
        DATE_TRUNC('month', order_date) AS month,
        SUM(total_amount) AS revenue
    FROM orders
    GROUP BY 1
)
SELECT
    month,
    revenue,
    LAG(revenue) OVER (ORDER BY month) AS prev_revenue,
    ROUND(
        100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
        / NULLIF(LAG(revenue) OVER (ORDER BY month), 0),
        2
    ) AS growth_pct
FROM monthly
ORDER BY month;

```

Q9. Find users who logged in on at least 3 consecutive days.

```

WITH login_days AS (
    SELECT DISTINCT user_id, DATE(login_time) AS login_date
    FROM user_logins
),
with_gaps AS (
    SELECT
        user_id,
        login_date,
        login_date - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY
login_date))::int AS grp
    FROM login_days
)
SELECT user_id, MIN(login_date), MAX(login_date), COUNT(*) AS consecutive_days
FROM with_gaps
GROUP BY user_id, grp
HAVING COUNT(*) >= 3;

```

Q10. Pivot: Show each product's monthly sales as columns (Jan-Dec).

```

SELECT
    product_id,
    SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 1 THEN amount END) AS jan,
    SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 2 THEN amount END) AS feb,
    SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 3 THEN amount END) AS mar,
    SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 4 THEN amount END) AS apr,
    SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 5 THEN amount END) AS may,
    SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 6 THEN amount END) AS jun,
    SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 7 THEN amount END) AS jul,
    SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 8 THEN amount END) AS aug,
    SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 9 THEN amount END) AS sep,
    SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 10 THEN amount END) AS oct,
    SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 11 THEN amount END) AS nov,
    SUM(CASE WHEN EXTRACT(MONTH FROM order_date) = 12 THEN amount END) AS dec
FROM order_items
JOIN orders USING (order_id)
GROUP BY product_id;

```

Q11. Find pairs of products frequently bought together (market basket).

```

SELECT
    a.product_id AS product_a,
    b.product_id AS product_b,
    COUNT(*) AS times_together
FROM order_items a
JOIN order_items b ON a.order_id = b.order_id AND a.product_id < b.product_id
GROUP BY a.product_id, b.product_id
HAVING COUNT(*) >= 10
ORDER BY times_together DESC
LIMIT 20;

```

Q12. Calculate the median order value per customer.

```

SELECT
    customer_id,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY total_amount) AS median_order_value
FROM orders
GROUP BY customer_id;

```

Advanced (Hard / Amazon-Style)

Q13. Session analysis: calculate session duration, sessions start when gap > 30 minutes.

```

WITH events AS (
    SELECT
        user_id,
        event_time,
        LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) AS
prev_event_time
    FROM user_events

```

```

),
session_starts AS (
    SELECT
        user_id,
        event_time,
        SUM(CASE WHEN prev_event_time IS NULL
                OR event_time - prev_event_time > INTERVAL '30 minutes'
                THEN 1 ELSE 0 END)
            OVER (PARTITION BY user_id ORDER BY event_time) AS session_id
    FROM events
)
SELECT
    user_id,
    session_id,
    MIN(event_time) AS session_start,
    MAX(event_time) AS session_end,
    MAX(event_time) - MIN(event_time) AS session_duration
FROM session_starts
GROUP BY user_id, session_id;

```

Q14. Amazon Orders: find all orders that were delivered late (past promised date) and calculate the late rate per seller.

```

WITH late_orders AS (
    SELECT
        o.seller_id,
        COUNT(*) AS total_orders,
        SUM(CASE WHEN o.actual_delivery_date > o.promised_delivery_date THEN 1 ELSE
0 END) AS late_orders
    FROM orders o
    WHERE o.actual_delivery_date IS NOT NULL
    GROUP BY o.seller_id
)
SELECT
    seller_id,
    total_orders,
    late_orders,
    ROUND(100.0 * late_orders / NULLIF(total_orders, 0), 2) AS late_rate_pct
FROM late_orders
ORDER BY late_rate_pct DESC;

```

Q15. Calculate the 7-day moving average of daily active users.

```

WITH daily_dau AS (
    SELECT DATE(event_time) AS day, COUNT(DISTINCT user_id) AS dau
    FROM user_events
    GROUP BY 1
)
SELECT
    day,
    dau,

```

```

ROUND(AVG(dau) OVER (ORDER BY day ROWS BETWEEN 6 PRECEDING AND CURRENT ROW), 0)
AS moving_avg_7d
FROM daily_dau
ORDER BY day;

```

Q16. Find the first purchase category for each customer and their subsequent category changes.

```

WITH ordered_purchases AS (
    SELECT
        customer_id,
        category,
        order_date,
        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS
purchase_num
    FROM orders o
    JOIN order_items oi USING (order_id)
    JOIN products p USING (product_id)
),
first_purchase AS (
    SELECT customer_id, category AS first_category
    FROM ordered_purchases WHERE purchase_num = 1
)
SELECT
    fp.customer_id,
    fp.first_category,
    op.category AS subsequent_category,
    COUNT(*) AS times
FROM first_purchase fp
JOIN ordered_purchases op ON fp.customer_id = op.customer_id AND op.purchase_num > 1
WHERE op.category <> fp.first_category
GROUP BY fp.customer_id, fp.first_category, op.category;

```

Q17. Find "whale" customers — top 5% by LTV who contribute to 80% of revenue (Pareto).

```

WITH customer_ltv AS (
    SELECT
        customer_id,
        SUM(total_amount) AS ltv,
        SUM(SUM(total_amount)) OVER () AS total_revenue,
        NTILE(20) OVER (ORDER BY SUM(total_amount)) AS percentile_bucket
    FROM orders
    GROUP BY customer_id
),
cumulative AS (
    SELECT
        customer_id,
        ltv,
        total_revenue,
        SUM(ltv) OVER (ORDER BY ltv DESC) AS cumulative_revenue,
        SUM(ltv) OVER (ORDER BY ltv DESC) / total_revenue AS cumulative_pct
    FROM customer_ltv

```

```

)
SELECT customer_id, ltv, ROUND(100.0 * ltv / total_revenue, 4) AS revenue_share_pct
FROM cumulative
WHERE cumulative_pct <= 0.80
ORDER BY ltv DESC;

```

Q18. Inventory forecasting: predict stockout risk given current stock and average daily sales.

```

WITH daily_sales AS (
    SELECT
        product_id,
        AVG(daily_qty) AS avg_daily_sales
    FROM (
        SELECT product_id, DATE(created_at) AS sale_date, SUM(quantity) AS daily_qty
        FROM order_items
        WHERE created_at >= CURRENT_DATE - 90
        GROUP BY product_id, DATE(created_at)
    ) d
    GROUP BY product_id
)
SELECT
    i.product_id,
    i.quantity_on_hand,
    ds.avg_daily_sales,
    ROUND(i.quantity_on_hand / NULLIF(ds.avg_daily_sales, 0)) AS
days_until_stockout,
    CASE
        WHEN i.quantity_on_hand / NULLIF(ds.avg_daily_sales, 0) <= 7 THEN 'CRITICAL'
        WHEN i.quantity_on_hand / NULLIF(ds.avg_daily_sales, 0) <= 14 THEN 'WARNING'
        ELSE 'OK'
    END AS stockout_risk
FROM inventory i
JOIN daily_sales ds USING (product_id)
ORDER BY days_until_stockout ASC NULLS LAST;

```

Q19. Find "churned" customers (no purchase in 60 days) who came back (cohort re-engagement).

```

WITH purchase_gaps AS (
    SELECT
        customer_id,
        order_date,
        LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS
prev_order_date,
        order_date - LAG(order_date) OVER (PARTITION BY customer_id ORDER BY
order_date) AS gap_days
    FROM orders
)
SELECT
    customer_id,
    prev_order_date AS churned_after,
    order_date AS reactivation_date,

```

```

        gap_days
FROM purchase_gaps
WHERE gap_days > 60
ORDER BY gap_days DESC;

```

Q20. Build a cohort retention table: % of users from each signup month still active N months later.

```

WITH cohorts AS (
    SELECT user_id, DATE_TRUNC('month', signup_date) AS cohort_month
    FROM users
),
activity AS (
    SELECT DISTINCT user_id, DATE_TRUNC('month', event_time) AS activity_month
    FROM user_events
),
cohort_activity AS (
    SELECT
        c.cohort_month,
        EXTRACT(YEAR FROM AGE(a.activity_month, c.cohort_month)) * 12 +
        EXTRACT(MONTH FROM AGE(a.activity_month, c.cohort_month)) AS month_number,
        COUNT(DISTINCT c.user_id) AS active_users
    FROM cohorts c
    JOIN activity a ON c.user_id = a.user_id
    WHERE a.activity_month >= c.cohort_month
    GROUP BY c.cohort_month, month_number
),
cohort_sizes AS (
    SELECT cohort_month, COUNT(*) AS cohort_size
    FROM cohorts GROUP BY cohort_month
)
SELECT
    ca.cohort_month,
    cs.cohort_size,
    ca.month_number,
    ca.active_users,
    ROUND(100.0 * ca.active_users / cs.cohort_size, 1) AS retention_pct
FROM cohort_activity ca
JOIN cohort_sizes cs USING (cohort_month)
ORDER BY ca.cohort_month, ca.month_number;

```

Q21. Detect anomalies: products whose price changed more than 20% day-over-day.

```

WITH price_changes AS (
    SELECT
        product_id,
        price_date,
        price,
        LAG(price) OVER (PARTITION BY product_id ORDER BY price_date) AS prev_price
    FROM product_price_history
)
SELECT

```

```

product_id,
price_date,
prev_price,
price,
ROUND(100.0 * (price - prev_price) / NULLIF(prev_price, 0), 2) AS pct_change
FROM price_changes
WHERE ABS((price - prev_price) / NULLIF(prev_price, 0)) > 0.20;

```

Q22. Find the longest streak of daily active users (max consecutive days).

```

WITH daily AS (
    SELECT DISTINCT user_id, DATE(event_time) AS active_date
    FROM events
),
numbered AS (
    SELECT user_id, active_date,
           active_date - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY
active_date))::int AS island
    FROM daily
),
streaks AS (
    SELECT user_id, island, COUNT(*) AS streak_len,
           MIN(active_date) AS start_date, MAX(active_date) AS end_date
    FROM numbered
    GROUP BY user_id, island
)
SELECT user_id, MAX(streak_len) AS longest_streak
FROM streaks
GROUP BY user_id
ORDER BY longest_streak DESC;

```

Q23. Attribution modeling: last-touch attribution — credit the last marketing channel before purchase.

```

WITH last_touch AS (
    SELECT DISTINCT ON (o.order_id)
           o.order_id,
           o.customer_id,
           o.total_amount,
           e.channel
    FROM orders o
    JOIN marketing_events e ON e.customer_id = o.customer_id
                           AND e.event_time < o.order_date
    ORDER BY o.order_id, e.event_time DESC
)
SELECT
    channel,
    COUNT(*) AS attributed_orders,
    SUM(total_amount) AS attributed_revenue
FROM last_touch

```

```
GROUP BY channel
ORDER BY attributed_revenue DESC;
```

Q24. Find products that are consistently in the bottom 10% of sales across all months (chronic underperformers).

```
WITH monthly_sales AS (
    SELECT
        product_id,
        DATE_TRUNC('month', order_date) AS month,
        SUM(quantity) AS units_sold,
        NTILE(10) OVER (PARTITION BY DATE_TRUNC('month', order_date) ORDER BY
SUM(quantity)) AS decile
    FROM order_items
    JOIN orders USING (order_id)
    GROUP BY product_id, DATE_TRUNC('month', order_date)
)
SELECT product_id, COUNT(*) AS months_in_bottom_10pct
FROM monthly_sales
WHERE decile = 1
GROUP BY product_id
HAVING COUNT(*) = (SELECT COUNT(DISTINCT DATE_TRUNC('month', order_date)) FROM
orders)
ORDER BY months_in_bottom_10pct DESC;
```

Q25. Write a recursive CTE to find all subordinates of a given manager (org chart traversal).

```
WITH RECURSIVE org_tree AS (
    -- Base case: the starting manager
    SELECT employee_id, name, manager_id, 0 AS depth
    FROM employees
    WHERE employee_id = :manager_id -- parameter

    UNION ALL

    -- Recursive case: direct reports at each level
    SELECT e.employee_id, e.name, e.manager_id, ot.depth + 1
    FROM employees e
    JOIN org_tree ot ON e.manager_id = ot.employee_id
)
SELECT employee_id, name, depth,
       REPEAT(' ', depth) || name AS org_chart
FROM org_tree
ORDER BY depth, name;
```

4. Database Design Tasks

Task 1: Design Amazon Order Management System

Prompt: Design the database schema for Amazon's order management system supporting millions of orders/day.

```
-- Core schema
CREATE TABLE customers (
  customer_id    BIGSERIAL PRIMARY KEY,
  email          VARCHAR(255) UNIQUE NOT NULL,
  name           VARCHAR(200),
  created_at     TIMESTAMPTZ DEFAULT NOW(),
  country_code   CHAR(2)
);

CREATE TABLE addresses (
  address_id     BIGSERIAL PRIMARY KEY,
  customer_id    BIGINT REFERENCES customers(customer_id),
  address_type   VARCHAR(20) CHECK (address_type IN ('shipping','billing')),
  street         VARCHAR(500),
  city           VARCHAR(100),
  state          VARCHAR(100),
  postal_code    VARCHAR(20),
  country_code   CHAR(2)
);

CREATE TABLE sellers (
  seller_id      BIGSERIAL PRIMARY KEY,
  name           VARCHAR(200) NOT NULL,
  rating         NUMERIC(3,2) CHECK (rating BETWEEN 0 AND 5),
  country_code   CHAR(2)
);

CREATE TABLE products (
  product_id     BIGSERIAL PRIMARY KEY,
  seller_id      BIGINT REFERENCES sellers(seller_id),
  title          VARCHAR(500) NOT NULL,
  category_id    INT,
  weight_kg      NUMERIC(8,3),
  is_active      BOOLEAN DEFAULT TRUE,
  created_at     TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE product_prices (
  price_id       BIGSERIAL PRIMARY KEY,
  product_id     BIGINT REFERENCES products(product_id),
  price          NUMERIC(10,2) NOT NULL,
  currency       CHAR(3) DEFAULT 'USD',
  valid_from     TIMESTAMPTZ NOT NULL,
  valid_to       TIMESTAMPTZ
);

CREATE TABLE orders (
  order_id       BIGSERIAL PRIMARY KEY,
```

```

customer_id    BIGINT REFERENCES customers(customer_id),
shipping_address_id BIGINT REFERENCES addresses(address_id),
status         VARCHAR(30) NOT NULL DEFAULT 'pending',
total_amount   NUMERIC(12,2),
currency       CHAR(3) DEFAULT 'USD',
placed_at      TIMESTAMPTZ DEFAULT NOW(),
promised_delivery TIMESTAMPTZ
) PARTITION BY RANGE (placed_at);

CREATE TABLE order_items (
    order_item_id BIGSERIAL,
    order_id       BIGINT NOT NULL,
    product_id     BIGINT REFERENCES products(product_id),
    quantity       INT NOT NULL CHECK (quantity > 0),
    unit_price     NUMERIC(10,2) NOT NULL,
    seller_id      BIGINT REFERENCES sellers(seller_id),
    PRIMARY KEY (order_item_id, order_id)
) PARTITION BY RANGE (order_id);

-- Partitioning orders by month
CREATE TABLE orders_2024_01 PARTITION OF orders
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

-- Indexes
CREATE INDEX idx_orders_customer ON orders (customer_id, placed_at DESC);
CREATE INDEX idx_orders_status ON orders (status) WHERE status NOT IN ('delivered', 'cancelled');
CREATE INDEX idx_order_items_order ON order_items (order_id);
CREATE INDEX idx_order_items_product ON order_items (product_id);

```

Discussion Points:

- Partitioning by `placed_at` allows pruning old data
- Partial index on `status` for active orders only
- `order_items` denormalizes `seller_id` to avoid joins in common queries
- `product_prices` as separate table supports price history

Task 2: Design a Product Catalog with Full-Text Search

```

CREATE TABLE product_catalog (
    product_id    BIGSERIAL PRIMARY KEY,
    title         VARCHAR(500) NOT NULL,
    description    TEXT,
    brand         VARCHAR(200),
    tags          TEXT[],
    attributes    JSONB,
    search_vector  TSVECTOR GENERATED ALWAYS AS (
        SETWEIGHT(TO_TSVECTOR('english', COALESCE(title, '')), 'A') ||
        SETWEIGHT(TO_TSVECTOR('english', COALESCE(brand, '')), 'B') ||
        SETWEIGHT(TO_TSVECTOR('english', COALESCE(description, '')), 'C')
    ) STORED
)

```

```

);

CREATE INDEX idx_product_search ON product_catalog USING GIN (search_vector);
CREATE INDEX idx_product_attributes ON product_catalog USING GIN (attributes);
CREATE INDEX idx_product_tags ON product_catalog USING GIN (tags);

-- Query
SELECT product_id, title, ts_rank(search_vector, query) AS rank
FROM product_catalog, PLAINTO_TSQUERY('english', 'wireless bluetooth headphones')
query
WHERE search_vector @@ query
ORDER BY rank DESC
LIMIT 20;

```

Task 3: Design Inventory Management with Real-Time Stock

```

CREATE TABLE warehouses (
    warehouse_id SERIAL PRIMARY KEY,
    name VARCHAR(200),
    location POINT, -- PostGIS for geo
    is_active BOOLEAN DEFAULT TRUE
);

CREATE TABLE inventory (
    inventory_id BIGSERIAL PRIMARY KEY,
    product_id BIGINT NOT NULL,
    warehouse_id INT NOT NULL,
    quantity_on_hand INT NOT NULL DEFAULT 0 CHECK (quantity_on_hand >= 0),
    quantity_reserved INT NOT NULL DEFAULT 0 CHECK (quantity_reserved >= 0),
    last_updated TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE (product_id, warehouse_id)
);

CREATE TABLE inventory_transactions (
    txn_id BIGSERIAL PRIMARY KEY,
    product_id BIGINT NOT NULL,
    warehouse_id INT NOT NULL,
    txn_type VARCHAR(20) CHECK (txn_type IN
('receive', 'reserve', 'release', 'ship', 'adjust')),
    quantity_delta INT NOT NULL,
    reference_id BIGINT, -- order_id, receipt_id, etc.
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Reserve stock atomically
CREATE OR REPLACE FUNCTION reserve_inventory(
    p_product_id BIGINT,
    p_warehouse_id INT,
    p_quantity INT
) RETURNS BOOLEAN AS $$

```

```

DECLARE
    v_available INT;
BEGIN
    SELECT quantity_on_hand - quantity_reserved INTO v_available
    FROM inventory
    WHERE product_id = p_product_id AND warehouse_id = p_warehouse_id
    FOR UPDATE;

    IF v_available >= p_quantity THEN
        UPDATE inventory
        SET quantity_reserved = quantity_reserved + p_quantity,
            last_updated = NOW()
        WHERE product_id = p_product_id AND warehouse_id = p_warehouse_id;
        RETURN TRUE;
    END IF;
    RETURN FALSE;
END;
$$ LANGUAGE plpgsql;

```

Task 4: Design a Seller Performance Metrics System

```

CREATE TABLE seller_metrics_daily (
    metric_id      BIGSERIAL PRIMARY KEY,
    seller_id      BIGINT NOT NULL,
    metric_date    DATE NOT NULL,
    orders_received INT DEFAULT 0,
    orders_shipped INT DEFAULT 0,
    orders_late    INT DEFAULT 0,
    orders_cancelled INT DEFAULT 0,
    total_revenue  NUMERIC(14,2) DEFAULT 0,
    avg_rating     NUMERIC(3,2),
    defect_rate    NUMERIC(5,4),
    UNIQUE (seller_id, metric_date)
);

-- Rollup view for monthly
CREATE MATERIALIZED VIEW seller_metrics_monthly AS
SELECT
    seller_id,
    DATE_TRUNC('month', metric_date) AS month,
    SUM(orders_received) AS orders_received,
    SUM(orders_shipped) AS orders_shipped,
    SUM(orders_late) AS orders_late,
    ROUND(SUM(orders_late)::numeric / NULLIF(SUM(orders_shipped), 0), 4) AS
late_shipment_rate,
    SUM(total_revenue) AS total_revenue,
    AVG(avg_rating) AS avg_rating
FROM seller_metrics_daily
GROUP BY seller_id, DATE_TRUNC('month', metric_date)
WITH DATA;

```

```
CREATE UNIQUE INDEX ON seller_metrics_monthly (seller_id, month);
```

Task 5: Design a Recommendation Engine Schema

```
CREATE TABLE user_item_interactions (  
  interaction_id BIGSERIAL,  
  user_id       BIGINT NOT NULL,  
  product_id    BIGINT NOT NULL,  
  interaction_type VARCHAR(20) CHECK (interaction_type IN  
( 'view', 'cart', 'purchase', 'review' )),  
  interaction_weight NUMERIC(3,1), -- view=1, cart=3, purchase=5  
  occurred_at      TIMESTAMPTZ DEFAULT NOW(),  
  PRIMARY KEY (interaction_id, occurred_at)  
) PARTITION BY RANGE (occurred_at);  
  
CREATE TABLE product_similarity (  
  product_a BIGINT NOT NULL,  
  product_b BIGINT NOT NULL,  
  similarity_score NUMERIC(6,5) NOT NULL,  
  algorithm VARCHAR(20), -- 'cosine', 'jaccard'  
  computed_at TIMESTAMPTZ DEFAULT NOW(),  
  PRIMARY KEY (product_a, product_b)  
);  
  
CREATE TABLE user_recommendations (  
  user_id       BIGINT NOT NULL,  
  product_id    BIGINT NOT NULL,  
  score         NUMERIC(8,5),  
  reason        VARCHAR(50), -- 'collab_filter', 'content_based', 'trending'  
  expires_at    TIMESTAMPTZ,  
  PRIMARY KEY (user_id, product_id)  
);  
  
CREATE INDEX idx_rec_user_score ON user_recommendations (user_id, score DESC);
```

5. System Design with DB Focus

Scenario 1: Design Amazon's Cart Service (Black Friday Scale)

Requirements: 100M concurrent users, cart must persist across sessions, checkout in < 1 second.

Architecture Decision:

Write Path:

User Action → API Gateway → Cart Service →
Primary: DynamoDB (low-latency writes, key=user_id)
Async: SQS → Lambda → Aurora PostgreSQL (audit/analytics)

Read Path:

Cart Read → ElastiCache Redis (TTL 24h) → DynamoDB fallback

Checkout Path:

Redis WATCH + MULTI/EXEC for atomic cart → order conversion

PostgreSQL: orders table with SELECT ... FOR UPDATE on inventory

PostgreSQL Role:

- Order confirmation (ACID required)
- Inventory reservation (row-level locking)
- Analytics on abandoned carts
- Compliance/audit trail

Key Indexes:

```
CREATE INDEX idx_orders_customer_status ON orders (customer_id, status, placed_at
DESC);
CREATE INDEX idx_inventory_product_avail ON inventory (product_id)
WHERE quantity_on_hand > quantity_reserved;
```

Scenario 2: Design Order Tracking System

Requirements: 500M order events/day, customers need real-time updates.

Data Model:

```
CREATE TABLE order_events (
  event_id          BIGSERIAL,
  order_id          BIGINT NOT NULL,
  event_type        VARCHAR(50) NOT NULL,
  event_data        JSONB,
  carrier_code      VARCHAR(10),
  tracking_number   VARCHAR(100),
  occurred_at       TIMESTAMPTZ NOT NULL
) PARTITION BY RANGE (occurred_at);

-- Time-series style with BRIN index
CREATE INDEX idx_events_time_brin ON order_events USING BRIN (occurred_at);
CREATE INDEX idx_events_order ON order_events (order_id, occurred_at DESC);
```

Scale Considerations:

- Kinesis → Lambda → PostgreSQL pipeline for event ingestion
- BRIN index on occurred_at (1/100th size of B-tree for append-only data)
- Partition by month, keep 6 months hot, archive to S3 via pg_partman

Scenario 3: Design Fraud Detection Data Pipeline

Requirements: Detect fraudulent transactions in real-time (< 100ms), historical analysis for model training.

Schema:

```

CREATE TABLE transactions (
  txn_id          UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  customer_id     BIGINT NOT NULL,
  amount         NUMERIC(10,2) NOT NULL,
  currency        CHAR(3),
  merchant_id     BIGINT,
  device_id       VARCHAR(100),
  ip_address      INET,
  occurred_at     TIMESTAMPTZ DEFAULT NOW(),
  fraud_score     NUMERIC(5,4),
  is_fraud        BOOLEAN
);

-- Fast lookups for rules engine
CREATE INDEX idx_txn_customer_recent ON transactions (customer_id, occurred_at
DESC);
CREATE INDEX idx_txn_ip ON transactions (ip_address, occurred_at DESC);
CREATE INDEX idx_txn_device ON transactions (device_id, occurred_at DESC);

-- Rule: flag if customer spent > 3x their average in one transaction
SELECT t.txn_id, t.amount,
       AVG(t2.amount) AS avg_amount,
       t.amount / NULLIF(AVG(t2.amount), 0) AS amount_ratio
FROM transactions t
JOIN transactions t2 ON t.customer_id = t2.customer_id
                    AND t2.occurred_at BETWEEN NOW() - INTERVAL '30 days' AND
t.occurred_at
WHERE t.occurred_at >= NOW() - INTERVAL '1 hour'
GROUP BY t.txn_id, t.amount
HAVING t.amount > 3 * AVG(t2.amount);

```

6. Behavioral Questions (LP-Mapped)

Customer Obsession

- "Tell me about a time you redesigned a database schema to improve user experience."
- "Describe a situation where you had to balance technical debt vs. customer-facing features in DB design."
- "When have you sacrificed performance optimization to ship faster for customers?"

Ownership

- "Tell me about a production database incident you owned end-to-end. How did you handle it?"
- "Describe a time you took ownership of a database migration that wasn't technically your responsibility."

Dive Deep

- "Walk me through debugging a slow PostgreSQL query. What was the execution plan? What did you change?"
- "Tell me about a time you found a non-obvious root cause in a data issue."

Bias for Action

- "Tell me about a time you made a schema change with incomplete information. What did you do?"
- "Describe a time you wrote a quick-fix SQL before implementing the proper solution."

Invent and Simplify

- "Tell me about a time you simplified an overly complex query or schema."
- "Describe a database design that was creative or unconventional."

Frugality

- "How have you reduced database costs through schema or query changes?"
- "Tell me about optimizing storage or compute costs on a database system."

7. Mock Interview Script

Opening (5 min)

Interviewer: "Tell me briefly about your database experience."

Strong Answer: "I've spent 4 years working with PostgreSQL in production — managing a 2TB database for an e-commerce platform with 10M monthly active users. I've handled schema migrations, index optimization, query tuning, replication setup, and led a major OLTP-to-Aurora migration. I'm comfortable with both the SQL layer and PostgreSQL internals like MVCC, WAL, and the planner."

SQL Round (20 min)

Interviewer: "Given a `user_sessions` table with columns `user_id`, `session_start`, `session_end`, find users who had overlapping sessions."

Think-aloud approach:

1. "I need to self-join on `user_id` where sessions overlap."
2. "Overlap condition: `start_a < end_b AND start_b < end_a`"

```
SELECT DISTINCT a.user_id
FROM user_sessions a
JOIN user_sessions b ON a.user_id = b.user_id
                     AND a.session_id <> b.session_id
                     AND a.session_start < b.session_end
                     AND b.session_start < a.session_end;
```

Interviewer: "Now how would you make this fast for 100M rows?"

Answer: "I'd add a GiST index using `tsrange` :

```
ALTER TABLE user_sessions ADD COLUMN session_range tsrange
GENERATED ALWAYS AS (tsrange(session_start, session_end)) STORED;
CREATE INDEX idx_sessions_range ON user_sessions USING GIST (user_id,
session_range);
```

The overlap operator `&&` on `tsrange` uses the GiST index efficiently."

Design Round (20 min)

Interviewer: "Design the database for a flash sale system — 10,000 items, 1M users competing to buy in 60 seconds."

Strong Response Structure:

- 1. "The core challenge is preventing overselling under extreme concurrency."
- 2. Propose optimistic locking with version columns
- 3. Discuss SKIP LOCKED for queue-based processing
- 4. Show the inventory deduction pattern:

```
-- Flash sale inventory with advisory lock
BEGIN;
SELECT pg_advisory_xact_lock(product_id); -- serialize per product

UPDATE flash_sale_inventory
SET quantity_available = quantity_available - 1,
    version = version + 1
WHERE product_id = $1
    AND quantity_available > 0
    AND sale_id = $2
RETURNING order_slot_number;

COMMIT;
```

8. Evaluation Rubric

SDE I / Junior Data Engineer

Criteria	Pass	Bar
Basic SQL	Correct JOINS, GROUP BY, HAVING	Gets right answer, may need hints
Schema Design	Identifies entities, creates FKs	May miss indexes
LPs	Can tell 2-3 stories	Stories are concrete
System Design	High-level components	Not expected to go deep

SDE II / Mid-Level

Criteria	Pass	Bar
SQL	Window functions, CTEs, subqueries	Writes optimal queries without prompting
Schema Design	Normalization, indexing strategy	Discusses trade-offs proactively
LPs	5-6 stories, STAR format	S/T is concise, A/R is concrete
System Design	Identifies bottlenecks, suggests scaling	Discusses CAP trade-offs

DB Internals	MVCC, vacuum, execution plans	Can read EXPLAIN ANALYZE
--------------	-------------------------------	--------------------------

SDE III / Senior

Criteria	Exceeds	Bar
SQL	Complex multi-step, edge cases handled	Considers NULL handling, overflow
Schema Design	Partitioning, sharding, denormalization trade-offs	Frames every decision with scale
LPs	Drives the conversation, impacts L-level above	Shows influence beyond team
System Design	Full end-to-end, failure modes, runbooks	Discusses SLAs, alerting, DR
Mentorship	Has mentored others on DB topics	Can discuss teaching others

9. Red Flags

- Cannot explain what an execution plan shows
- Writes queries that work but ignores NULL edge cases
- Schema design with no discussion of indexing
- LP stories that lack measurable outcomes ("I improved performance" — by how much?)
- Cannot distinguish OLTP from OLAP workloads
- Conflates PostgreSQL-specific features with ANSI SQL (or doesn't know the difference)
- Cannot explain why a query is slow or how to fix it
- Zero awareness of transactions, isolation levels, or locking

10. Green Flags

- Proactively mentions trade-offs before being asked
- Uses EXPLAIN ANALYZE naturally in schema/query discussions
- Thinks about failure modes in design ("what if this insert fails mid-transaction?")
- LP stories show specific metrics: "reduced query time from 8s to 200ms" or "cut storage by 40%"
- Asks clarifying questions before designing ("what's the read/write ratio? how many rows?")
- Understands the difference between page cache hits and index hits
- Mentions monitoring and observability in design (pg_stat_statements, slow query logs)
- Shows awareness of Aurora PostgreSQL vs. vanilla PostgreSQL differences

11. Salary Bands (Approximate, USD, 2024–2025)

Level	Role	Base	RSU (4yr)	Sign-On	Total Comp
SDE I	Junior DB Engineer	\$140K–\$165K	\$60K–\$100K	\$20K–\$30K	\$170K–\$210K
SDE II	Database Engineer	\$165K–\$195K	\$100K–\$180K	\$30K–\$50K	\$210K–\$280K
SDE III	Senior DB Engineer	\$195K–\$230K	\$200K–\$350K	\$50K–\$80K	\$290K–\$420K
Principal	Principal Engineer	\$230K–\$280K	\$400K–\$700K	\$80K–\$120K	\$450K–\$700K

Seattle/Bay Area rates. New York ~5–10% lower. Austin/remote ~15–20% lower.

12. Preparation Timeline (4-Week Plan)

Week 1: SQL Foundations & Amazon Context

Day	Task
Mon	Study Amazon LP framework — write 5 STAR stories for DB topics
Tue	Practice 10 medium LeetCode SQL problems
Wed	Study Aurora PostgreSQL vs. RDS PostgreSQL differences
Thu	Review window functions: ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, NTILE
Fri	Mock SQL interview (time yourself: 2 problems in 45 min)
Sat	Study partitioning, indexes, EXPLAIN ANALYZE
Sun	Review LP stories; refine with metrics

Week 2: Database Design & System Design

Day	Task
Mon	Design e-commerce schema from scratch (timed, 45 min)
Tue	Study Amazon system design (order management, inventory, recommendations)
Wed	Practice CTEs, recursive queries, complex aggregations
Thu	Study DynamoDB vs. PostgreSQL trade-offs (when Amazon would use each)
Fri	Mock design interview with a peer
Sat	Study pg_stat_statements, pg_stat_activity, autovacuum
Sun	Practice 5 hard SQL problems

Week 3: Amazon-Specific Depth

Day	Task
Mon	Deep dive: Aurora Read Replicas and how they differ from PG streaming replication
Tue	Study Redshift for analytics (DISTKEY, SORTKEY, columnar storage)
Wed	Practice LP storytelling: Ownership, Dive Deep, Bias for Action
Thu	Study AWS DMS for database migrations (common for Amazon interviews)
Fri	Full mock loop (4 rounds, 45 min each) with a peer
Sat	Review weak areas from mock

Sun	Rest and light review
-----	-----------------------

Week 4: Final Polish

Day	Task
Mon	Review all LP stories; tighten metrics
Tue	Hard SQL problems: sessions, gaps & islands, cohorts
Wed	Mock behavioral round (Bar Raiser style)
Thu	Review system design patterns: CQRS, event sourcing with DB
Fri	Light review + rest
Sat	Final review of key concepts, no new material
Interview Day	Early start, review 3–5 LP stories, stay calm